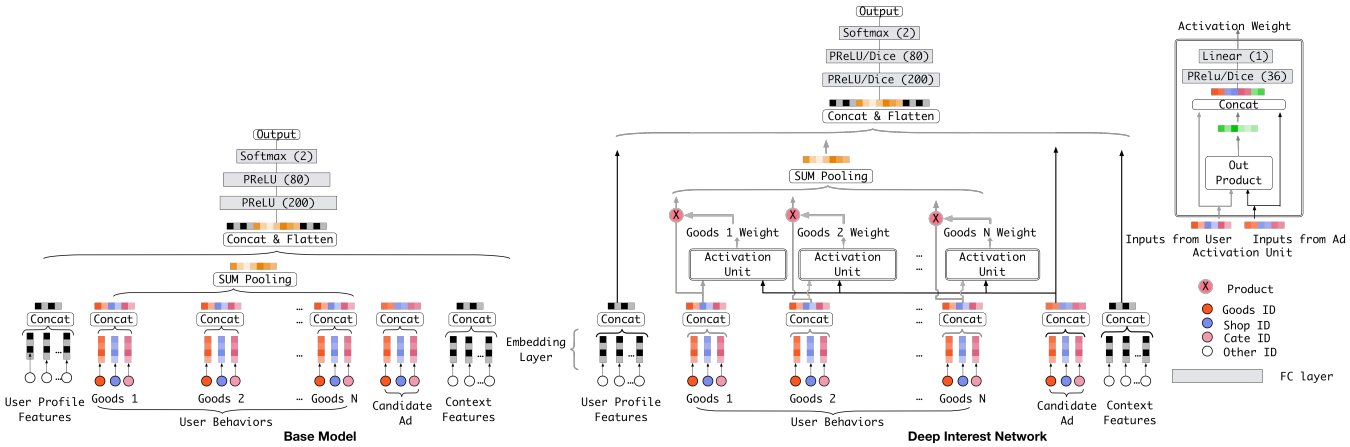

Figure 2: Network Architecture. The left part illustrates the network of base model (Embedding&MLP). Embeddings of cate_id, shop_id and goods_id belong to one goods are concatenated to represent one visited goods in user's behaviors. Right part is our proposed DIN model. It introduces a local activation unit, with which the representation of user interests varies adaptively given different candidate ads.

Two most commonly used pooling layers are sum pooling and average pooling, which apply element-wise sum/average operations to the list of embedding vectors.

Both embedding and pooling layers operate in a group-wise manner, mapping the original sparse features into multiple fixed-length representation vectors. Then all the vectors are concatenated together to obtain the overall representation vector for the instance.

MLP. Given the concatenated dense representation vector, fully connected layers are used to learn the combination of features automatically. Recently developed methods [4, 5, 10] focus on designing structures of MLP for better information extraction.

Loss. The objective function used in base model is the negative log-likelihood function defined as:

$$L = -\frac{1}{N} \sum_{(\boldsymbol{x}, y) \in \mathcal{S}} (y \log p(\boldsymbol{x}) + (1 - y) \log(1 - p(\boldsymbol{x}))), \tag{2}$$

where $\mathcal{S}$ is the training set of size N, with $\boldsymbol{x}$ as the input of the network and $y \in \{0, 1\}$ as the label, $p(\boldsymbol{x})$ is the output of the network after the softmax layer, representing the predicted probability of sample $\boldsymbol{x}$ being clicked.

4.3 The structure of Deep Interest Network

Among all those features of Table 1, user behavior features are critically important and play key roles in modeling user interests in the scenario of e-commerce applications.

Base model obtains a fixed-length representation vector of user interests by pooling all the embedding vectors over the user behavior feature group, as Eq.(1). This representation vector stays the same for a given user, in regardless of what candidate ads are. In this way, the user representation vector with a limited dimension will be a bottleneck to express user's diverse interests. To make it capable enough, an easy method is to expand the dimension of embedding vector, which unfortunately will increase the size of learning parameters heavily. It will lead to overfitting under limited training data and add the burden of computation and storage, which may not be tolerated for an industrial online system.

Is there an elegant way to represent user's diverse interests in one vector under limited dimension? The local activation characteristic of user interests gives us inspiration to design a novel model named deep interest network(DIN). Imagine when the young mother mentioned above in section 3 visits the e-commerce site, she finds the displayed new handbag cute and clicks it. Let's dissect the driving force of click action. The displayed ad hits the related interests of this young mother by soft-searching her historical behaviors and finding that she had browsed similar goods of tote bag and leather handbag recently. In other words, behaviors related to displayed ad greatly contribute to the click action. DIN simulates this process by paying attention to the representation of locally activated interests w.r.t. given ad. Instead of expressing all user's diverse interests with the same vector, DIN adaptively calculate the representation vector of user interests by taking into consideration the relevance of historical behaviors w.r.t. candidate ad. This representation vector varies over different ads.

The right part of Fig.2 illustrates the architecture of DIN. Compared with base model, DIN introduces a novel designed local activation unit and maintains the other structures the same. Specifically, activation units are applied on the user behavior features, which performs as a weighted sum pooling to adaptively calculate user representation $\boldsymbol{v}_U$ given a candidate ad A, as shown in Eq.(3)

$$\boldsymbol{v}_U(A) = f(\boldsymbol{v}_A, \boldsymbol{e}_1, \boldsymbol{e}_2, .., \boldsymbol{e}_H) = \sum_{j=1}^{H} a(\boldsymbol{e}_j, \boldsymbol{v}_A)\boldsymbol{e}_j = \sum_{j=1}^{H} \boldsymbol{w}_j \boldsymbol{e}_j, \tag{3}$$

where $\{\boldsymbol{e}_1, \boldsymbol{e}_2, ..., \boldsymbol{e}_H\}$ is the list of embedding vectors of behaviors of user U with length of H, $\boldsymbol{v}_A$ is the embedding vector of ad A. In this way, $\boldsymbol{v}_U(A)$ varies over different ads. $a(\cdot)$ is a feed-forward network with output as the activation weight, as illustrated in Fig.2. Apart from the two input embedding vectors, $a(\cdot)$ adds the out

product of them to feed into the subsequent network, which is an explicit knowledge to help relevance modeling.

Local activation unit of Eq.(3) shares similar ideas with attention methods which are developed in NMT task[1]. However different from traditional attention method, the constraint of $\sum_i w_i = 1$ is relaxed in Eq.(3), aiming to reserve the intensity of user interests. That is, normalization with softmax on the output of $a(\cdot)$ is abandoned. Instead, value of $\sum_i w_i$ is treated as an approximation of the intensity of activated user interests to some degree. For example, if one user's historical behaviors contain 90% clothes and 10% electronics. Given two candidate ads of T-shirt and phone, T-shirt activates most of the historical behaviors belonging to clothes and may get larger value of $\boldsymbol{v}_U$ (higher intensity of interest) than phone. Traditional attention methods lose the resolution on the numerical scale of $\boldsymbol{v}_U$ by normalizing of the output of $a(\cdot)$.

We have tried LSTM to model user historical behavior data in the sequential manner. But it shows no improvement. Different from text which is under the constraint of grammar in NLP task, the sequence of user historical behaviors may contain multiple concurrent interests. Rapid jumping and sudden ending over these interests causes the sequence data of user behaviors to seem to be noisy. A possible direction is to design special structures to model such data in a sequence way. We leave it for future research.

5 TRAINING TECHNIQUES

In the advertising system in Alibaba, numbers of goods and users scale up to hundreds of millions. Practically, training industrial deep networks with large scale sparse input features is of great challenge. In this section, we introduce two important techniques which are proven to be helpful in practice.

5.1 Mini-batch Aware Regularization

Overfitting is a critical challenge for training industrial networks. For example, with addition of fine-grained features, such as features of *goods_ids* with dimensionality of 0.6 billion (including *visited_goods_ids* of user and *goods_id* of ad as described in Table 1), model performance falls rapidly after the first epoch during training without regularization, as the dark green line shown in Fig.4 in later section 6.5. It is not practical to directly apply traditional regularization methods, such as ℓ_2 and ℓ_1 regularization, on training networks with sparse inputs and hundreds of millions of parameters. Take ℓ_2 regularization as an example. Only parameters of non-zero sparse features appearing in each mini-batch needs to be updated in the scenario of SGD based optimization methods without regularization. However, when adding ℓ_2 regularization it needs to calculate L2-norm over the whole parameters for each mini-batch, which leads to extremely heavy computations and is unacceptable with parameters scaling up to hundreds of millions.

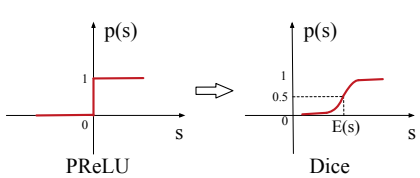

Figure 3: Control function of PReLU and Dice.

In this paper, we introduce an efficient mini-batch aware regularizer, which only calculates the L2-norm over the parameters of sparse features appearing in each mini-batch and makes the computation possible. In fact, it is the embedding dictionary that contributes most of the parameters for CTR networks and arises the difficulty of heavy computation. Let $\mathbf{W} \in \mathbb{R}^{D \times K}$ denote parameters of the whole embedding dictionary, with D as the dimensionality of the embedding vector and K as the dimensionality of feature space. Expand the ℓ_2 regularization on $\mathbf{W}$ over samples

$$L_2(\mathbf{W}) = \|\mathbf{W}\|_2^2 = \sum_{j=1}^{K} \|\boldsymbol{w}_j\|_2^2 = \sum_{(\boldsymbol{x},y)\in\mathcal{S}} \sum_{j=1}^{K} \frac{I(\boldsymbol{x}_j \neq 0)}{n_j} \|\boldsymbol{w}_j\|_2^2, \tag{4}$$

where $\boldsymbol{w}_j \in \mathbb{R}^D$ is the j-th embedding vector, $I(\boldsymbol{x}_j \neq 0)$ denotes if the instance $\boldsymbol{x}$ has the feature id j, and n_j denotes the number of occurrence for feature id j in all samples. Eq.(4) can be transformed into Eq.(5) in the mini-batch aware manner

$$L_2(\mathbf{W}) = \sum_{j=1}^{K} \sum_{m=1}^{B} \sum_{(\boldsymbol{x},y)\in\mathcal{B}_m} \frac{I(\boldsymbol{x}_j \neq 0)}{n_j} \|\boldsymbol{w}_j\|_2^2, \tag{5}$$

where B denotes the number of mini-batches, $\mathcal{B}_m$ denotes the m-th mini-batch. Let $\alpha_{mj} = \max_{(\boldsymbol{x},y)\in\mathcal{B}_m} I(\boldsymbol{x}_j \neq 0)$ denote if there is at least one instance having the feature id j in mini-batch $\mathcal{B}_m$. Then Eq.(5) can be approximated by

$$L_2(\mathbf{W}) \approx \sum_{j=1}^{K} \sum_{m=1}^{B} \frac{\alpha_{mj}}{n_j} \|\boldsymbol{w}_j\|_2^2. \tag{6}$$

In this way, we derive an approximated mini-batch aware version of ℓ_2 regularization. For the m-th mini-batch, the gradient w.r.t. the embedding weights of feature j is

$$\boldsymbol{w}_j \leftarrow \boldsymbol{w}_j - \eta \left[\frac{1}{|\mathcal{B}_m|} \sum_{(\boldsymbol{x},y)\in\mathcal{B}_m} \frac{\partial L(p(\boldsymbol{x}), y)}{\partial \boldsymbol{w}_j} + \lambda \frac{\alpha_{mj}}{n_j} \boldsymbol{w}_j \right], \tag{7}$$

in which only parameters of features appearing in m-th mini-batch participate in the computation of regularization.

5.2 Data Adaptive Activation Function

PReLU [12] is a commonly used activation function

$$f(s) = \begin{cases} s & \text{if } s > 0 \\ \alpha s & \text{if } s \leq 0. \end{cases} = p(s) \cdot s + (1 - p(s)) \cdot \alpha s, \tag{8}$$

where s is one dimension of the input of activation function $f(\cdot)$ and $p(s) = I(s > 0)$ is an indicator function which controls $f(s)$ to switch between two channels of $f(s) = s$ and $f(s) = \alpha s$. α in the second channel is a learning parameter. Here we refer to $p(s)$ as the control function. The left part of Fig.3 plots the control function of PReLU. PReLU takes a hard rectified point with value of 0, which may be not suitable when the inputs of each layer follow different distributions. Take this into consideration, we design a novel data adaptive activation function named ***Dice***,

$$f(s) = p(s) \cdot s + (1 - p(s)) \cdot \alpha s, \quad p(s) = \frac{1}{1 + e^{-\frac{s - E[s]}{\sqrt{Var[s] + \epsilon}}}} \tag{9}$$

with the control function to be plotted in the right part of Fig.3. In the training phrase, $E[s]$ and $Var[s]$ is the mean and variance of input in each mini-batch. In the testing phrase, $E[s]$ and $Var[s]$ is calculated by moving averages $E[s]$ and $Var[s]$ over data. ϵ is a small constant which is set to be 10^{-8} in our practice.

Dice can be viewed as a generalization of PReLu. The key idea of Dice is to adaptively adjust the rectified point according to distribution of input data, whose value is set to be the mean of input. Besides, Dice controls smoothly to switch between the two channels. When $E(s) = 0$ and $Var[s] = 0$, Dice degenerates into PReLU.

6 EXPERIMENTS

In this section, we present our experiments in detail, including datasets, evaluation metric, experimental setup, model comparison and the corresponding analysis. Experiments on two public datasets with user behaviors as well as a dataset collected from the display advertising system in Alibaba demonstrate the effectiveness of proposed approach which outperforms state-of-the-art methods on the CTR prediction task. Both the public datasets and experiment codes are made available[1].

6.1 Datasets and Experimental Setup

Amazon Dataset[2]. Amazon Dataset contains product reviews and metadata from Amazon, which is used as benchmark dataset[13, 18, 23]. We conduct experiments on a subset named Electronics, which contains 192,403 users, 63,001 goods, 801 categories and 1,689,188 samples. User behaviors in this dataset are rich, with more than 5 reviews for each users and goods. Features include goods_id, cate_id, user reviewed goods_id_list and cate_id_list. Let all behaviors of a user be $(b_1, b_2, \ldots, b_k, \ldots, b_n)$, the task is to predict the (k+1)-th reviewed goods by making use of the first k reviewed goods. Training dataset is generated with $k = 1, 2, \ldots, n-2$ for each user. In the test set, we predict the last one given the first $n-1$ reviewed goods. For all models, we use SGD as the optimizer with exponential decay, in which learning rate starts at 1 and decay rate is set to 0.1. The mini-batch size is set to be 32.

MovieLens Dataset[3]. MovieLens data[11] contains 138,493 users, 27,278 movies, 21 categories and 20,000,263 samples. To make it suitable for CTR prediction task, we transform it into a binary classification data. Original user rating of the movies is continuous value ranging from 0 to 5. We label the samples with rating of 4 and 5 to be positive and the rest to be negative. We segment the data into training and testing dataset based on userID. Among all 138,493 users, of which 100,000 are randomly selected into training set (about 14,470,000 samples) and the rest 38,493 into the test set (about 5,530,000 samples). The task is to predict whether user will rate a given movie to be above 3(positive label) based on historical behaviors. Features include movie_id, movie_cate_id and user rated movie_id_list, movie_cate_id_list. We use the same optimizer, learning rate and mini-batch size as described on Amazon Dataset.

Alibaba Dataset. We collected traffic logs from the online display advertising system in Alibaba, of which two weeks' samples are used for training and samples of the following day for testing. The size of training and testing set is about 2 billions and 0.14 billion respectively. For all the deep models, the dimensionality of embedding vector is 12 for the whole 16 groups of features. Layers of MLP is set by $192 \times 200 \times 80 \times 2$. Due to the huge size of data, we set the mini-batch size to be 5000 and use Adam[15] as the optimizer. We apply exponential decay, in which learning rate starts at 0.001 and decay rate is set to 0.9.

[2]http://jmcauley.ucsd.edu/data/amazon/

[3]https://grouplens.org/datasets/movielens/20m/

Table 2: Statistics of datasets used in this paper.

Dataset	Users	Goods[a]	Categories	Samples
Amazon(Electro).	192,403	63,001	801	1,689,188
MovieLens.	138,493	27,278	21	20,000,263
Alibaba.	60 million	0.6 billion	100,000	2.14 billion

[a] For MovieLens dataset, goods refer to be movies.

The statistics of all the above datasets is shown in Table 2. Volume of Alibaba Dataset is much larger than both Amazon and MovieLens, which brings more challenges.

6.2 Competitors

- **LR[19]**. Logistic regression (LR) is a widely used shallow model before deep networks for CTR prediction task. We implement it as a weak baseline.
- **BaseModel**. As introduced in section4.2, BaseModel follows the Embedding&MLP architecture and is the base of most of subsequently developed deep networks for CTR modeling. It acts as a strong baseline for our model comparison.
- **Wide&Deep[4]**. In real industrial applications, Wide&Deep model has been widely accepted. It consists of two parts: i) wide model, which handles the manually designed cross product features, ii) deep model, which automatically extracts nonlinear relations among features and equals to the BaseModel. Wide&Deep needs expertise feature engineering on the input of the "wide" module. We follow the practice in [10] to take cross-product of user behaviors and candidates as wide inputs. For example, in MovieLens dataset, it refers to the cross-product of user rated movies and candidate movies.
- **PNN[5].** PNN can be viewed as an improved version of BaseModel by introducing a product layer after embedding layer to capture high-order feature interactions.
- **DeepFM[10]**. It imposes a factorization machines as "wide" module in Wide&Deep saving feature engineering jobs.

6.3 Metrics

In CTR prediction field, AUC is a widely used metric[8]. It measures the goodness of order by ranking all the ads with predicted CTR, including intra-user and inter-user orders. An variation of user weighted AUC is introduced in [7, 13] which measures the goodness of intra-user order by averaging AUC over users and is shown to be more relevant to online performance in display advertising system. We adapt this metric in our experiments. For simplicity, we still refer it as AUC. It is calculated as follows:

$$\text{AUC} = \frac{\sum_{i=1}^{n} \#impression_i \times \text{AUC}_i}{\sum_{i=1}^{n} \#impression_i}, \tag{10}$$

where n is the number of users, $\#impression_i$ and AUC_i are the number of impressions and AUC corresponding to the i-th user.

Besides, we follow [25] to introduce RelaImpr metric to measure relative improvement over models. For a random guesser, the value of AUC is 0.5. Hence RelaImpr is defined as below:

$$RelaImpr = \left(\frac{\text{AUC(measured model)} - 0.5}{\text{AUC(base model)} - 0.5} - 1 \right) \times 100\%. \tag{11}$$